SIAG

Societal Impact Action Group

IIT Madras Alumni Association

Application Documentation & User Guide

Streamlit + MySQL · CRUD for Members, Projects, Media & Documents

Prepared by the SIAG Technical Team

Contents

What this document covers and where to find it

How to use this document

Start with Section 1 (Architecture) to understand the app's layout. Section 2 explains who can do what. Section 3 walks through every page — read the one you need. Section 5 is the single reference for getting the app running from scratch. Section 6 is a reference for developers extending the codebase.

1. Overview & Architecture

A Streamlit application backed by MySQL, organized as one app.py entry point plus a pages/ directory of route modules and a utils/ directory of shared helpers.

SIAG is a web application for the Societal Impact Action Group of the IIT Madras Alumni Association. It gives administrators, editors, and members a single place to manage people, projects, media, and documents — all backed by a MySQL database and served through Streamlit's native navigation system.

Key design choices:

- **Streamlit navigation (st.navigation)** — seven pages declared in app.py and rendered as a sidebar menu. Each page imports only the utilities it needs.
- **Two session types — user and member.** Admins/editors log in through the users table (username + bcrypt password). Members log in through the members table (email + bcrypt password). Both sessions are held in st.session_state under separate keys.
- **Row-level activity logging.** Every create, update, delete, login, logout, and purge event is written to the activity_log table via the log_activity() helper.
- **File storage on disk.** Uploaded media and documents are saved under uploads/media/ and uploads/documents/ with UUID-based filenames; the database stores the file path.
- **Environment-driven config.** No hardcoded secrets — all database and path settings come from a .env file loaded via python-dotenv.

Architecture diagram

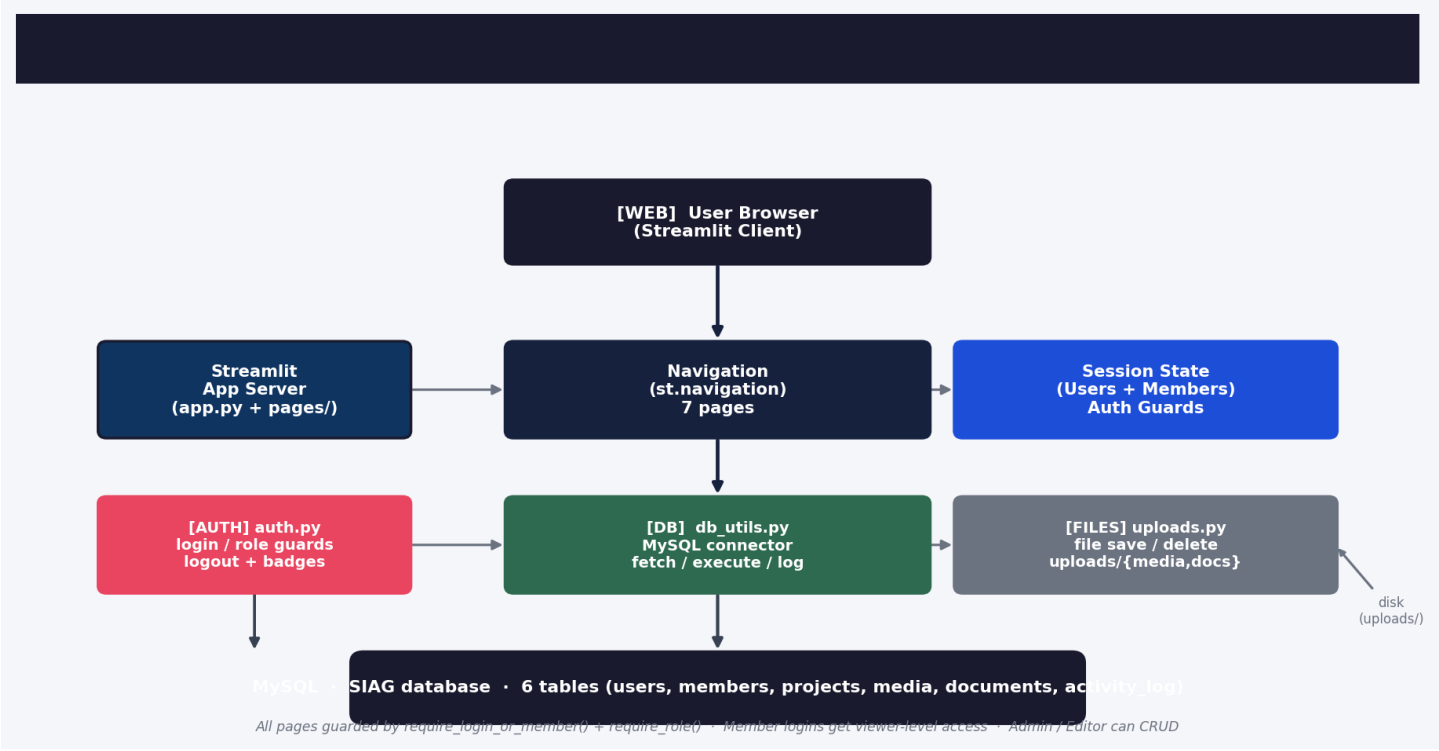

Figure 1 — SIAG architecture: browser, Streamlit server, auth/db/uploads layers, and MySQL. Arrows show request flow and dependency direction.

Request flow — end to end

1. User visits the app → **app.py** builds a `st.navigation` with seven pages and renders the sidebar.
2. Each page calls **require_login_or_member()** (from utils/auth.py) on entry. If neither session key is set, the login form is rendered inline and `st.stop()` halts further rendering.

3 Once a session exists, the page calls **require_role()** if it needs admin/editor-only access. Member sessions count as viewer-level.

4 Data operations go through **utils/db_utils.py** — `fetch_all()`, `fetch_one()`, and `execute_write()` — which open a MySQL connection per call via mysql-connector-python.

5 Uploads go through **utils/uploads.py**: `save_upload()` writes the file to disk with a UUID filename and returns the path; `delete_file()` removes it.

6 Side effects (creates, updates, deletes) are logged via `log_activity()` into the activity_log table before the page reruns.

Two parallel login paths

User login (admin/editor/viewer) — uses the **users** table. Session stored under `siag_session_user`. Identified by username.

Member login — uses the **members** table. Session stored under `siag_session_member`. Identified by email.

Either session grants access to protected pages. Role guards treat member sessions as viewers.

2. Authentication & User Roles

Two login paths, bcrypt password verification, session keys, and role-based guards.

Login paths

The app supports two distinct login paths, orchestrated in `utils/auth.py`. Both render inline on the calling page (no page switch — Streamlit's `st.switch_page` is incompatible with `st.navigation`).

Path	Table	Identifier	Session key	Roles
User login (admin/editor/viewer)	users	username	siag_session_user	admin, editor, viewer
Member login	members	email	siag_session_member	viewer (treated as viewer-level by guards)

Password verification

Passwords are hashed with bcrypt at rest. Both login paths follow the same pattern:

```
# User login (from utils/auth.py _render_user_form) user = fetch_one( "SELECT id, username, role, full_name, password_hash FROM users WHERE username = %s", (username,) ) if user and bcrypt.checkpw(password.encode(), user['password_hash'].encode()): st.session_state[SESSION_KEY] = {...} log_activity(user['id'], 'login', 'users', user['id'], ...) st.rerun()
```

```
# Member login (from utils/auth.py _render_member_form) member = fetch_one( "SELECT id, first_name, last_name, email, password_hash FROM members WHERE email = %s", (email,) ) if member and member.get('password_hash') and bcrypt.checkpw(password.encode(), member['password_hash'].encode()): st.session_state[MEMBER_SESSION_KEY] = {...} log_activity(None, 'login', 'members', member['id'], ...) st.rerun()
```

Default admin account

Seed account (from db_schema.sql)

Username: **admin**

Password: **siag2024**

Role: **admin**

The password hash in db_schema.sql is a placeholder — replace it with a real bcrypt hash for production. Use Python to generate one: `bcrypt.hashpw(b'siag2024', bcrypt.gensalt()).decode()`

Role-based access control

The `require_role(roles)` guard enforces permissions. Roles are strings: `'admin'`, `'editor'`, `'viewer'`. Member logins are treated as viewer-level — they gain access when `'viewer'` is in the allowed set, and are denied otherwise.

Role	Login path	Dashboard	Members	Projects	Media	Documents	Activity Log
admin	users	view	full CRUD + set password	full CRUD	upload/edit/delete	upload/edit/delete	view + purge
editor	users	view	full CRUD	full CRUD	upload/edit/delete	upload/edit/delete	view

Role	Login path	Dashboard	Members	Projects	Media	Documents	Activity Log
viewer	users	view	view only	view only	view only	view only	view
member	members	view	view only	view only	view only	view only	view

Session lifecycle

- **Setting a session:** login stores a dict in `st.session_state` under the session key and calls `st.rerun()`.
- **Reading a session:** `_get_current_user()` and `_get_current_member()` read from session state.
- **Clearing a session:** logout removes both keys and reruns — the page guard detects empty sessions and renders the login form.
- **Persistence:** sessions survive page-to-page navigation within one browser session but are lost on a full page reload if cookies are cleared (Streamlit session state behavior).

3. Page-by-Page Guide

Each page, who can access it, what it shows, and what actions are available.

Page overview

Page	File	Access	What it does
Dashboard	pages/01_Dashboard.py	any logged-in user	KPI cards (members, active projects, media, documents) + recent activity feed.
Members	pages/02_Members.py	any logged-in user (view); admin/editor (CRUD)	Searchable member list. Admin/editor can add, edit, delete members and set member passwords.
Projects	pages/03_Projects.py	any logged-in user (view); admin/editor (CRUD)	Project list with status, location, budget, outcome. Admin/editor can add, edit, delete.
Media Gallery	pages/04_Media.py	any logged-in user (view); admin/editor (upload/edit/delete)	Photo and video gallery. Upload, caption, tag, relate to projects/members.
Documents	pages/05_Documents.py	any logged-in user (view); admin/editor (upload/edit/delete)	PDF, doc, spreadsheet library. Upload, describe, tag, relate. Inline PDF preview + download.
Activity Log	pages/06_Activity_Log.py	any logged-in user (view); admin (purge)	Timestamped audit trail of all logged actions. Admin can purge the entire log.
Login	pages/00_Login.py	public (no session)	Shown when no session exists. Dual-tab login: admin + member.

3.1 Dashboard

The home page after login. It shows four KPI cards computed from the database and a reverse-chronological activity feed (20 most recent entries).

KPI cards

- **Members** — total count from the members table.
- **Active Projects** — count where status = 'Active'.
- **Media Items** — total count from the media table.
- **Documents** — total count from the documents table.

Recent activity

Fetches the 20 most recent activity_log rows joined to users (for username). Each entry is rendered as: *timestamp* `username` — action on entity_type: details.

3.2 Members

The members page is the most feature-rich. It has two zones: a searchable member list (visible to everyone) and an add/edit form plus a 'Set Member Password' form (admin/editor only).

```
# Access control in pages/02_Members.py (lines 7-8) require_login_or_member() is_admin_or_editor = _get_current_user() is not None and _get_current_user().get('role') in ('admin','editor')
```

Search

A text input strips and lowercases the query, then searches first_name, last_name, and email with a LIKE clause. Results are ordered by created_at DESC and shown as expandable cards.

Add / edit member

- **First Name and Last Name are required** — the save button is disabled unless both are present.
- **Password is optional on add; optional on edit (leave blank to keep current).** When provided, the form hashes it with bcrypt before writing.
- **created_by and updated_by** are set to the current user's id on create and update respectively.
- Every save logs an activity entry (create/update) and reruns the page.

Set member password

A separate form below the divider lists all members as a selectbox. Choosing one and entering a new password hashes it and updates the member's password_hash. A success message is shown via query_params (status=password_updated) and cleared on the next load.

Delete

The delete button (visible only to admin/editor) issues a DELETE and logs the action, then shows a success toast and reruns. No confirmation dialog — the action is immediate.

3.3 Projects

Projects are shown as expandable cards with full detail: category, village, district, state, status, lead, budget, dates, description, and outcome summary. A subquery counts related project_updates for each project.

- **Search** matches project_name, village, and district (case-insensitive LIKE).
- **Featured projects** are prefixed with a star (★) in the card title.
- **Edit** opens a form pre-filled from the existing row; status uses a selectbox with the current value selected by index.
- **Delete** removes the project (cascade deletes related project_updates).
- **Start/end dates** use Streamlit date_input, stored as DATE in MySQL.
- **Budget** is a number_input stored as DECIMAL(15,2).

> **INSERT statement note**
>
> The INSERT in pages/03_Projects.py (line 114-116) sets created_by and updated_by to the current user's id. The project_updates table is not populated from this form — updates are added separately if that feature is implemented.

3.4 Media Gallery

The media page has three sections: an upload form (admin/editor), an edit form (admin/editor, when an editing_media_id is set), and the gallery grid.

Upload

- **Title is required** in addition to a file.
- **File types**: jpg, jpeg, png, gif, mp4, mov.
- **Related to** can be general, project, or member — with an optional related_id.
- The file is saved to uploads/media/ with a UUID name; the path is stored in the media table.
- Tags are stored as a comma-separated string.

Gallery

- Photos render via `st.image()` with use_column_width=True.
- Videos render via `st.video()`.
- Each card shows title, media type (uppercased in the header), caption, tags, and related info.

- Admin/editor sees Edit and Delete buttons per item.

Edit media

The edit form pre-fills all fields from the existing row. If a new file is uploaded, the old file is deleted from disk first (delete_file). If no new file is uploaded, the existing path is retained. The related_type is normalized to lowercase-English forms.

3.5 Documents

Documents support PDF, doc, docx, xls, xlsx, txt, and csv. The page structure mirrors media: upload form, edit form, and a list with per-item actions.

Upload

- **Title is required** plus a file.
- **File size (KB)** is computed from the uploaded file's byte length and stored in file_size_kb.
- **File type** is taken from file.type (MIME) and stored as-is.
- **Version** defaults to '1.0' (from the schema default).

Download + preview

- **Download button** — reads the file from disk and offers it via st.download_button for all types.
- **Inline PDF viewer** — for PDFs, the file contents are base64-encoded and embedded in an `iframe` via unsafe_allow_html. Non-PDF files skip this step.
- If the file is missing from disk, a warning is shown and download/preview are skipped.

Edit document

The edit form mirrors the upload form. Replacing the file triggers a delete of the old file and a save of the new one. Metadata (title, description, tags, related_type, related_id) can be updated independently of the file. After a successful update, the editing state is cleared via a two-part reset (_clear_edit flag + editing_doc_id = None).

3.6 Activity Log

The log shows the 500 most recent entries joined to users for username and full_name. It is rendered as a Streamlit dataframe (st.dataframe, stretch width).

- **Admin only:** a 'Purge Activity Log' button appears above the table. It deletes all rows and logs the purge event itself (so the action is auditable).
- **Viewer/member:** read-only access to the table.
- **Logged actions:** login, logout, create, update, delete, purge — across users, members, projects, media, documents, and activity_log.

4. Database Schema

SIAG database — six tables, foreign keys, and indexes. All SQL in db_schema.sql.

Run `db_schema.sql` against your MySQL instance to create the `SIAG` database and all tables. The schema uses InnoDB, utf8mb4, and AUTO_INCREMENT primary keys throughout.

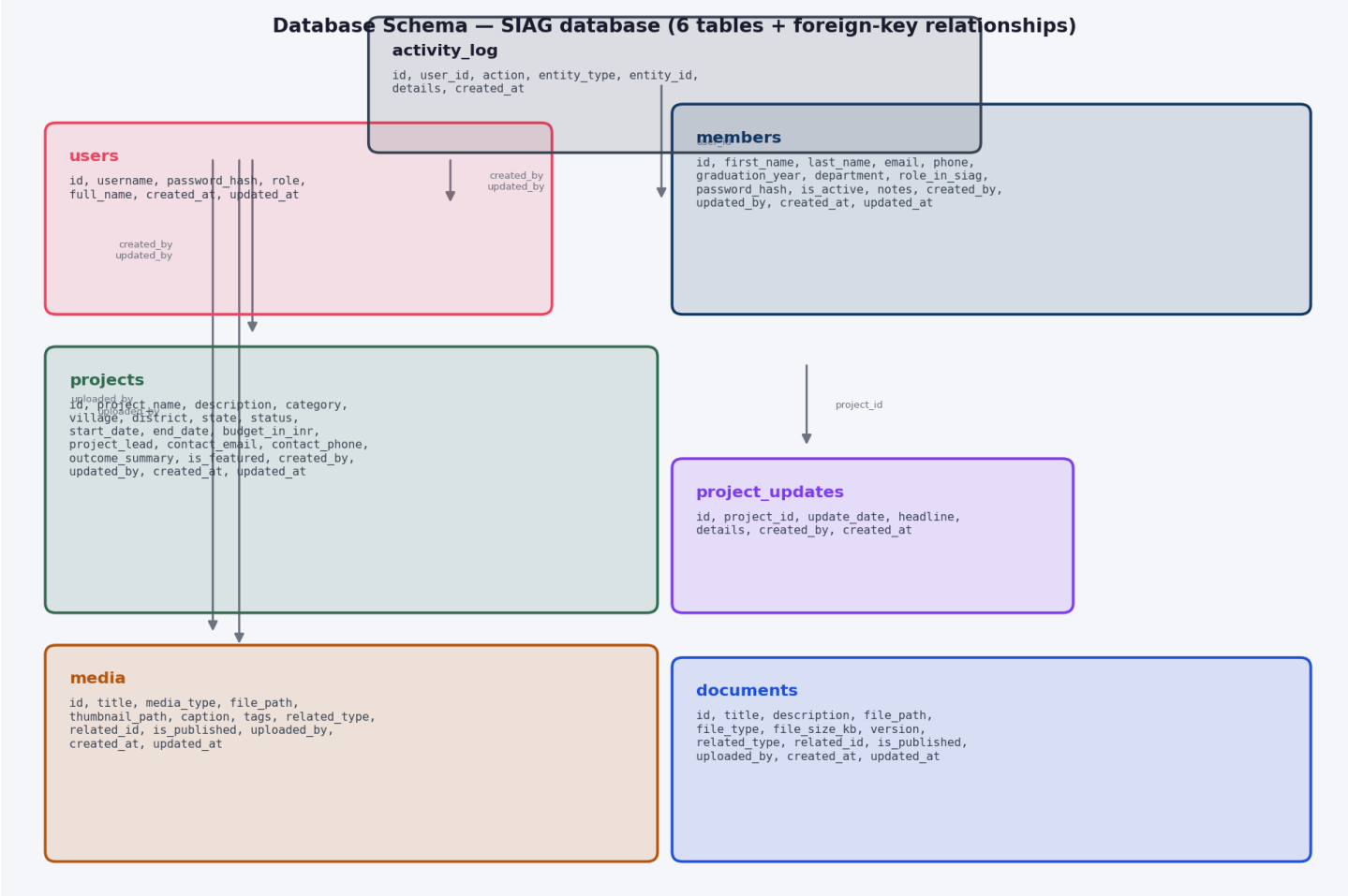

Figure 2 — Database schema: seven table boxes (including project_updates) with foreign-key arrows. users is the central parent for created_by/updated_by/uploaded_by.user_id.

Table reference

4.1 users — application users (admin, editor, viewer)

Column	Type	Notes
id	INT AUTO_INCREMENT PK	Primary key.
username	VARCHAR(100) UNIQUE NOT NULL	Login identifier. Unique.
password_hash	VARCHAR(255) NOT NULL	Bcrypt hash of the user's password.
role	ENUM('admin','editor','viewer')	Default: 'viewer'. Controls page-level access.
full_name	VARCHAR(200)	Display name; shown in the user badge.
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	Auto-set on insert.
updated_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Auto-updated.

4.2 members — SIAG member records (separate login path)

Column	Type	Notes
id	INT AUTO_INCREMENT PK	Primary key.
first_name	VARCHAR(150) NOT NULL	Required.
last_name	VARCHAR(150) NOT NULL	Required.
email	VARCHAR(200)	Login identifier for member login. Used in the password-set form.
phone	VARCHAR(30)	Optional contact.
graduation_year	INT	Optional; min 0, max 2099 in the form.
department	VARCHAR(100)	Optional.
role_in_siag	VARCHAR(100)	Optional role label (shown in the member card).
password_hash	VARCHAR(255)	Bcrypt hash. NULL if the member has no password set.
is_active	BOOLEAN DEFAULT TRUE	Active flag; indexed.
notes	TEXT	Free-form notes.
created_by	INT	FK → users.id. Set on insert by the current user.
updated_by	INT	FK → users.id. Set on update by the current user.
created_at / updated_at	TIMESTAMP	Auto timestamps.

FK on created_by / updated_by → users(id) ON DELETE SET NULL.

4.3 projects — project records with location, budget, status

Column	Type	Notes
id	INT AUTO_INCREMENT PK	Primary key.
project_name	VARCHAR(250) NOT NULL	Required; displayed in card title.
description	TEXT	Optional long description.
category	VARCHAR(100)	Optional.
village / district / state	VARCHAR(150)	Location fields.
status	ENUM('Planning','Active','On Hold','Completed','Closed')	Default: 'Planning'. Indexed.
start_date / end_date	DATE	Optional project dates.
budget_in_inr	DECIMAL(15,2) DEFAULT 0.00	Budget in Indian rupees.
project_lead	VARCHAR(200)	Optional.
contact_email / contact_phone	VARCHAR(200) / VARCHAR(30)	Optional contact.
outcome_summary	TEXT	Optional outcome text.
is_featured	BOOLEAN DEFAULT FALSE	Featured projects get a ★ prefix.
created_by / updated_by	INT	FK → users.id.
created_at / updated_at	TIMESTAMP	Auto timestamps.

FK on created_by / updated_by → users(id) ON DELETE SET NULL. Indexes on status and category.

4.4 project_updates — timeline entries for projects

Column	Type	Notes
id	INT AUTO_INCREMENT PK	Primary key.
project_id	INT NOT NULL	FK → projects.id ON DELETE CASCADE.
update_date	DATE NOT NULL	Date of the update.
headline	VARCHAR(300)	Short title for the update.
details	TEXT	Longer update text.
created_by	INT	FK → users.id ON DELETE SET NULL.
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	Auto-set.

This table is defined in the schema but is not populated from any UI page yet — it is available for a future project-updates feature.

4.5 media — photos and videos

Column	Type	Notes
id	INT AUTO_INCREMENT PK	Primary key.
title	VARCHAR(250) NOT NULL	Required; card header.
media_type	ENUM('photo','video') DEFAULT 'photo'	Drives st.image vs st.video.
file_path	VARCHAR(500) NOT NULL	Absolute or relative path to the file on disk.
thumbnail_path	VARCHAR(500)	Optional thumbnail path (not used in current UI).
caption	TEXT	Optional caption shown under the image/video.
tags	VARCHAR(300)	Comma-separated tags.
related_type	ENUM('project','member','general') DEFAULT 'general'	What the media relates to.
related_id	INT	ID of the related project/member (NULL if general).
is_published	BOOLEAN DEFAULT FALSE	Published flag (not gated in current UI).
uploaded_by	INT	FK → users.id.
created_at / updated_at	TIMESTAMP	Auto timestamps.

Index on (media_type, is_published). Foreign key uploaded_by → users(id) ON DELETE SET NULL.

4.6 documents — PDF, Office, CSV, and text files

Column	Type	Notes
id	INT AUTO_INCREMENT PK	Primary key.
title	VARCHAR(250) NOT NULL	Required; card header + download filename base.
description	TEXT	Optional description shown in the card.
file_path	VARCHAR(500) NOT NULL	Path to the file on disk.
file_type	VARCHAR(50)	MIME type from the upload (e.g. 'application/pdf').
file_size_kb	INT	File size in kilobytes, computed on upload.
version	VARCHAR(20) DEFAULT '1.0'	Document version string.
related_type / related_id	ENUM / INT	Same pattern as media.
is_published	BOOLEAN DEFAULT FALSE	Published flag (not gated in current UI).
uploaded_by	INT	FK → users.id.
created_at / updated_at	TIMESTAMP	Auto timestamps.

Index on (related_type, related_id). Foreign key uploaded_by → users(id) ON DELETE SET NULL.

4.7 activity_log — audit trail

Column	Type	Notes
id	INT AUTO_INCREMENT PK	Primary key.
user_id	INT	FK → users.id ON DELETE SET NULL. NULL for member actions / purges.
action	VARCHAR(100) NOT NULL	One of: login, logout, create, update, delete, purge.
entity_type	VARCHAR(100) NOT NULL	users, members, projects, media, documents, activity_log.
entity_id	INT	ID of the affected entity (NULL for log-level actions).
details	TEXT	Human-readable detail (e.g. 'Updated John Doe').
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	Auto-set.

Index on (user_id, action). Every CRUD and auth operation calls log_activity().

5. Setup & Deployment

From an empty MySQL instance to a running Streamlit app — step by step.

5.1 Prerequisites

- **Python 3.10 or later** (the app was developed on Python 3.14).
- **MySQL 8.0+** (or a compatible MySQL-flavored server) accessible from the app host.
- **A database user** with CREATE, ALTER, INSERT, UPDATE, DELETE, and SELECT on the SIAG database (or the ability to run db_schema.sql as a privileged user).
- **Streamlit** — the app is a Streamlit application; run it with `streamlit run`.

5.2 Install dependencies

```
# requirements.txt streamlit==1.49.0 mysql-connector-python==9.4.0 Pillow==11.3.0
python-dotenv==1.2.1 python-multipart==0.0.20 bcrypt==4.3.0
```

Install with `pip install -r requirements.txt`. On some systems you may need `pip install --break-system-packages -r requirements.txt` if you are using the system Python without a virtual environment.

5.3 Configure the environment

Copy `.env.example` to `.env` and fill in the real values. The file is loaded by `python-dotenv` at import time in `utils/db_utils.py`, `utils/auth.py`, and `utils/uploads.py`.

```
# .env.example — copy to .env and edit DB_HOST=localhost DB_PORT=3306 DB_NAME=SIAG DB_USER=siag_user
DB_PASSWORD=your_password_here
```

Environment variable reference

Variable	Default	Used by	Notes
DB_HOST	localhost	db_utils.py	MySQL server hostname.
DB_PORT	3306	db_utils.py	MySQL server port (integer).
DB_NAME	SIAG	db_utils.py	Database name — must match the database created by db_schema.sql.
DB_USER	siag_user	db_utils.py	MySQL user with access to the SIAG database.
DB_PASSWORD	(none)	db_utils.py	MySQL password for DB_USER.
SIAG_UPLOAD_DIR	uploads	uploads.py	Root directory for uploaded files. Subfolders media/ and documents/ are created inside it.

Secrets

Never commit .env to version control. .gitignore in the project excludes it.

DB_PASSWORD and any future secret (e.g. a session secret) belong in .env only.

If you deploy to a server, set these as environment variables in the process launcher rather than relying on a .env file on disk.

5.4 Create the database and tables

```
# 1. Create the database (run once, as a MySQL user with CREATE privileges) mysql -u root -p < db_schema.sql # Or run the SQL interactively: # mysql -u root -p # CREATE DATABASE IF NOT EXISTS SIAG # CHARACTER SET utf8mb4 # COLLATE utf8mb4_unicode_ci; # USE SIAG; # -- then paste the rest of db_schema.sql
```

`db_schema.sql` is idempotent where possible: `CREATE DATABASE IF NOT EXISTS`, `CREATE TABLE IF NOT EXISTS`, and `INSERT ... ON DUPLICATE KEY UPDATE` for the admin seed. Re-running it against an existing database is safe for the schema — but the admin password hash line is a placeholder and should be replaced before re-running in production.

Seed admin account

Default admin (from db_schema.sql)

Username: **admin**

Password: **siag2024**

Role: **admin**

After first login, change the password via the member-password flow or by updating the users table directly with a new bcrypt hash.

5.5 Run the application

```
# From the project directory: cd /path/to/SIAG # Default port (8501): streamlit run app.py # Custom port: streamlit run app.py --server.port 8501 # Custom host (e.g. listen on all interfaces): streamlit run app.py --server.port 8501 --server.address 0.0.0.0
```

The app opens at `http://localhost:8501` by default. The first visit with no session renders the login page (pages/00_Login.py) inline.

Server settings reference

Flag	Default	Effect
--server.port	8501	TCP port to listen on.
--server.address	localhost	Interface to bind to. Use 0.0.0.0 to listen on all interfaces.
--server.headless	false	Run without opening a browser window (useful for background/service runs).
--browser.gatherUsageStats	true	Usage statistics; set false to disable.

5.6 File storage layout

Uploaded files live under the directory specified by `SIAG_UPLOAD_DIR` (default: `uploads`). The structure is:

```
uploads/ ■■■ media/ # photos (jpg, jpeg, png, gif) and videos (mp4, mov) ■ ■■■ . ■■■ documents/ # PDFs, Office docs, CSVs, text files ■ ■■■ . ■■■ thumbs/ # thumbnail directory (created but not yet used by the UI)
```

`utils/uploads.py` creates these subdirectories automatically on first upload via `ensure_dirs()`.

5.7 Logging and observability

- **Activity log:** every authenticated action is recorded in the activity_log table. View it on the Activity Log page. Admins can purge it.
- **Streamlit logs:** server-side logs go to stdout/stderr. Run with `streamlit run app.py 2>&1 | tee siag.log` to capture them.

- **No application-level log files:** the app does not write its own log files — all persistence is through the database activity_log table.

6. Project Structure

File tree, module responsibilities, and key functions.

File tree

```
SIAG/ ■■■ app.py # Entry point: page config + st.navigation + sidebar ■■■ requirements.txt # Python dependencies ■■■ .env # Environment variables (not committed) ■■■ .env.example # Template for .env ■■■ .gitignore # Excludes .env, __pycache__, etc. ■■■ db_schema.sql # CREATE DATABASE + all tables + seed admin ■■■ screen.png # Reference screenshot ■■■ uploads/ # File storage root (media/, documents/, thumbs/) ■■■ pages/ ■ ■■■ 00_Login.py # Login page (rendered when no session) ■ ■■■ 01_Dashboard.py # KPI cards + recent activity ■ ■■■ 02_Members.py # Member list + add/edit + set password ■ ■■■ 03_Projects.py # Project list + add/edit ■ ■■■ 04_Media.py # Media upload + gallery + edit ■ ■■■ 05_Documents.py # Document upload + list + download + PDF preview ■ ■■■ 06_Activity_Log.py # Activity log table + admin purge ■■■ utils/ ■■■ auth.py # Sessions, login forms, guards, badges, logout ■■■ db_utils.py # MySQL connection, query helpers, log_activity ■■■ uploads.py # File save/delete, directory setup
```

Module responsibilities

File	Responsibility	Key exports
app.py	Page configuration, navigation declaration, sidebar with logout and badges. Orchestrates the seven pages.	st.set_page_config, st.navigation, st.Page
pages/00_Login.py	Login page. Renders the login form inline when no session exists; otherwise shows an 'already logged in' notice and reruns.	render_login (from auth.py)
pages/01_Dashboard.py	Dashboard: KPI counts and recent activity feed.	require_login_or_member, require_role, fetch_all
pages/02_Members.py	Member CRUD, search, and set-password form.	require_login_or_member, fetch_all, fetch_one, execute_write, log_activity, save_upload, delete_file
pages/03_Projects.py	Project CRUD with subquery for update count.	require_login_or_member, require_role, fetch_all, fetch_one, execute_write, log_activity
pages/04_Media.py	Media upload, gallery, and edit.	require_login_or_member, require_role, fetch_all, fetch_one, execute_write, log_activity, save_upload, delete_file
pages/05_Documents.py	Document upload, list, download, inline PDF preview, edit.	require_login_or_member, require_role, fetch_all, fetch_one, execute_write, log_activity, save_upload, delete_file
pages/06_Activity_Log.py	Activity log viewer and admin purge.	require_login_or_member, require_role, fetch_all, execute_write, log_activity

File	Responsibility	Key exports
utils/auth.py	Session keys, login forms (user + member), role guards, badges, logout.	SESSION_KEY, MEMBER_SESSION_KEY, _get_current_user, _get_current_member, require_login_or_member, require_login, require_member_login, require_role, render_login, render_logout, render_current_user_badge, render_current_member_badge
utils/db_utils.py	MySQL connection, generic query helper, fetch_all, fetch_one, execute_write, log_activity.	get_config, get_connection, execute_query, fetch_all, fetch_one, execute_write, log_activity
utils/uploads.py	Upload directory setup, file save with UUID names, file deletion.	ensure_dirs, save_upload, delete_file, UPLOAD_ROOT, MEDIA_DIR, DOCS_DIR, THUMBS_DIR

Key functions — quick reference

utils/db_utils.py

- `get_config()` — returns a dict of DB connection parameters from env.
- `get_connection(dictionary=True)` — context manager that yields a mysql.connector connection and closes it on exit.
- `execute_query(query, params, fetch)` — generic helper: fetch='all' → list of dicts, 'one' → single dict or None, 'lastid' → lastrowid, 'none' → None.
- `fetch_all(query, params)` — shorthand for execute_query(fetch='all').
- `fetch_one(query, params)` — shorthand for execute_query(fetch='one').
- `execute_write(query, params)` — shorthand for execute_query(fetch='none').
- `log_activity(user_id, action, entity_type, entity_id, details)` — inserts a row into activity_log.

utils/auth.py

- `_get_current_user()` — returns the dict stored under siag_session_user, or None.
- `_get_current_member()` — returns the dict stored under siag_session_member, or None.
- `require_login_or_member()` — stops the page and renders the login form if neither session is set.
- `require_login()` — same as above (alias).
- `require_member_login()` — stops the page if no member session.
- `require_role(roles)` — checks the current user's role is in the set; member sessions count as viewer-level.
- `render_login()` — full login page render (used by pages/00_Login.py).
- `render_current_user_badge()` / `render_current_member_badge()` — sidebar captions.

utils/uploads.py

- `ensure_dirs()` — creates media/, documents/, thumbs/ under UPLOAD_ROOT.
- `save_upload(uploaded_file, subfolder)` — writes the file to the appropriate directory with a UUID name, returns the full path string.

- `delete_file(path_str)` — removes the file; silently ignores missing files.

7. Troubleshooting

Common issues, symptoms, and fixes.

7.1 Connection errors

MySQLInterfaceError / Access denied

Symptom: `Access denied for user 'xxx'@'localhost'` on page load.

Fix: check DB_USER and DB_PASSWORD in .env. Confirm the user exists in MySQL and has privileges on the SIAG database: `GRANT ALL PRIVILEGES ON SIAG.* TO 'user'@'host'; FLUSH PRIVILEGES;`

Check DB_HOST and DB_PORT — if MySQL is not on localhost:3306, set them accordingly.

Unknown database 'SIAG'

Symptom: `Unknown database 'SIAG'`.

Fix: run db_schema.sql against MySQL to create the database and tables.

Verify DB_NAME in .env matches the database you created (default: SIAG).

7.2 Login issues

- **Invalid username or password:** verify the username exists in the users table and the password hash matches. For the seed admin, the default password is `siag2024` — but only if the hash in db_schema.sql was replaced with the correct bcrypt hash for that password.
- **Member login fails:** confirm the member has a non-NULL password_hash in the members table. A NULL password_hash means no password has been set.
- **Login form not appearing:** if you see a protected page instead of the login form, the session may be stuck. Clear browser cookies / session storage and reload.

7.3 File upload issues

- **Permission denied on upload:** the process running Streamlit must have write permission to the uploads/ directory. Check ownership and permissions: `chmod -R 775 uploads/` and ensure the user matches.
- **File not found on disk (documents):** the download/preview code catches FileNotFoundError and shows a warning. This happens when a file was deleted outside the app or the path stored in the database is wrong.
- **Uploads not persisting across restarts:** confirm SIAG_UPLOAD_DIR points to a persistent location. If you run in a container or ephemeral environment, mount a volume.

7.4 App not starting

- **Module not found:** ensure all packages from requirements.txt are installed in the Python environment you are running Streamlit from.
- **Port already in use:** try a different port with `--server.port`.
- **Blank page / no rendering:** check the terminal for Python tracebacks. Streamlit prints errors to stdout/stderr; a missing .env or bad MySQL connection will surface there.

7.5 Role / permission issues

- **'You do not have permission to access this page':** the page called require_role() with a set that does not include your role. Admin and editor roles are stored in the users table. Member logins are treated as viewer-level.
- **Can view but cannot edit:** edit/delete/upload buttons are gated by `user_role in ('admin','editor')` checks in each page. Confirm your user's role in the users table.

7.6 Database schema changes

- **Adding a column:** ALTER TABLE the table, then update the corresponding page's SELECT and INSERT/UPDATE statements.
- **Re-running db_schema.sql:** idempotent for the schema, but the admin password hash line is a placeholder — replace it before re-running in production.
- **Indexes:** the schema creates indexes on is_active, status, category, (media_type, is_published), (related_type, related_id), and (user_id, action). Add more as query patterns evolve.

End of documentation. For changes or additions, edit the relevant page under pages/ or the utility under utils/, update db_schema.sql if the schema changes, and regenerate this PDF.